

PSE-METATRON-MONOLITH-01

Holistic Eigenmode Closure Layer

Formale Implementierungsspezifikation v0.1

Sebastian Klemm

Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

Zusammenfassung

Diese Spezifikation definiert einen additiven Closure-Layer fuer den bestehenden PSE-Stack. Der Layer fuehrt keine neue Engine ein, sondern verdichtet vorhandene NCTCS-, PhaseMatrix-, Dual-Fabric-, TPT/MTL-, Eval- und Validation-Artefakte in eine explizite holistische Eigenmode: den **MetatronOperator**. Der Operator ist keine zentrale Exekutivinstanz und kein privilegierter Steuerknoten. Er ist die deterministische, content-addressed und replayfaehige Projektion der globalen Systemkohärenz aus persistentem Feldtensor, MacroControlState, Trace, Conformance, spektraler Luecke, Mirror/Stitcher-Konsistenz und lokalen Monolith-Projektionen. Ziel ist ein eindeutiger Abschlusszustand pro validem Run: lokale Feldoperatoren koennen als isomorphe Projektionen der globalen Operatorik nachgewiesen werden, ohne dass lokale Resonanz mit Wahrheit oder Autoritaet verwechselt wird.

Inhaltsverzeichnis

1	Status und Ziel	3
1.1	Status	3
1.2	Ziel	3
1.3	Nicht-Ziele	3
2	Architekturposition	3
2.1	Einordnung	3
2.2	Semantische Umcodierung	4
3	Formaler Kern	4
3.1	Grundtuple	4
3.2	MetatronOperator	5
3.3	MonolithProjection	5
3.4	Isomorphe Feldoperator-Projektion	5
4	Kanonische Datentypen	5
4.1	Basistypen	5
4.2	MetatronRunDescriptor	6
4.3	HolisticEigenmodeState	6
4.4	LocalMonolithProjection	6
4.5	IsomorphicProjectionReport	7
4.6	SpectralGapStitchReport	7
5	Operatorpfad	7
5.1	Canonical Closure Chain	7
5.2	Referenzalgorithmus	7
6	Gates	8
6.1	Metatron Gate	8
6.2	Fail-Closed-Regel	8
6.3	Coherence-Is-Not-Truth-Regel	8
7	Conformance Classes	9
8	Integration mit Validation-Runner und Eval-Matrix	9
8.1	Runner-Erweiterung	9
8.2	Neue Reports	9
8.3	Neue Metriken	9
9	CLI	10
10	Tests	10
10.1	Unit Tests	10
10.2	Integration Tests	10
10.3	Negative Tests	10
11	Definition of Done	10
12	Coding-Agent-Auftrag	11
13	Schlussinvariante	11

1 Status und Ziel

1.1 Status

Dieses Dokument ist eine normative Implementierungsspezifikation. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich fuer Konformitaet.

1.2 Ziel

Der Layer schliesst eine semantisch-topologische Luecke im PSE-Stack: Nach NCTCS existiert ein gueltiger **MacroControlState**; nach PhaseMatrix existieren Zellen, Cluster, FunnelGraphs, FieldTensor-Stitches; nach TPT/MTL existieren topologische Orientierungsartefakte; nach Eval/Validation existieren Mess- und Replaybelege. Es fehlt jedoch ein expliziter Operator, der diese Artefakte zu einer globalen, auditierbaren, nicht-zentralistischen Eigenmode verdichtet.

Kerninvariante

Der **MetatronOperator** darf keine Struktur direkt erzwingen. Er darf nur aus bereits gegate-ten, replayfaehigen, evidence-linked, persistenten und konformitaetsgeprueften Artefakten einen **HolisticEigenmodeState** ableiten.

1.3 Nicht-Ziele

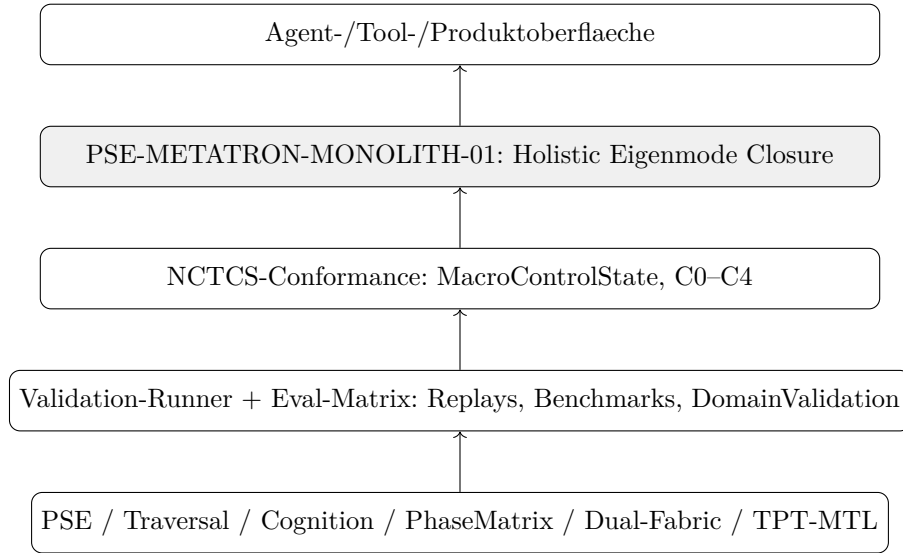
Die Spezifikation definiert nicht:

- einen zentralen Agenten;
- eine externe Aktorik;
- ein Autoritaets- oder Herrschaftsmodell;
- eine neue parallele PhaseMatrix- oder NCTCS-Engine;
- einen metaphysischen Wahrheitsclaim;
- eine Ersetzung der Eval-Matrix oder des Validation-Runners.

2 Architekturposition

2.1 Einordnung

Der Layer sitzt oberhalb von NCTCS-Conformance und Validation-Runner und unterhalb moeglicher Produkt-/Agentenoberflaechen.



2.2 Semantische Umcodierung

Die historischen Begriffe werden als technische Rollen verwendet:

Begriff	Normative technische Bedeutung
Monolith	Lokale irreversible Projektionsstelle, die einen gegate-ten Kandidaten in einen persistierbaren Trace-/Intent-/Commit-Kandidaten ueberfuehrt.
MetatronOperator	Globale Eigenmode-Projektion aus gueltiger Tensor-historie, MacroControlState, Conformance und lokalen Monolith-Projektionen.
IsomorphicProjection	Nachweis, dass ein lokaler Feldoperator dieselbe Operato-rik wie der globale Closure-Pfad abbildet.
HolisticSingularity	Kein Punktobjekt, sondern ein content-addressed Ab-schlusszustand der globalen Kohärenz nach Gates, Trace, Replay und Evaluation.
ImmanentOperator	Operator, der nur auf vorhandenen Systemartefakten ope-riert; kein externer Wahrheitsgeber.
SpectralGapStitch	Gated Stitch-Effekt, der eine messbare Stabilitaetsluecke vergroessert oder erhaelt.

3 Formaler Kern

3.1 Grundtuple

Eine gueltige Instanz dieses Layers ist das Tuple:

$$\mathcal{M}^+ = (K_0, F_T, M_{nctcs}, \mathcal{T}, \mathcal{L}, \mathcal{P}, \mathcal{I}, \Gamma, \mathcal{E})$$

mit:

- K_0 : NullCenter-Referenz aus NCTCS;
- F_T : persistenter FieldTensorState;
- M_{nctcs} : MacroControlState aus NCTCS-C4 oder niedrigerer Conformance-Klasse;
- $\mathcal{T}$: Trace-/Replay-/Evidence-Vertrag;

- $\mathcal{L}$: Menge lokaler Monolith-Projektionen;
- $\mathcal{P}$: Menge lokaler Feldoperatoren;
- $\mathcal{I}$: Isomorphie-Reports zwischen lokaler und globaler Operatorik;
- Γ : Stitch-/Seam-/SpectralGap-Artefakte;
- $\mathcal{E}$: Eval- und Validation-Artefakte.

3.2 MetatronOperator

Der `MetatronOperator` ist eine deterministische Abbildung:

$$\mathfrak{M} : (K_0, F_T, M_{nctcs}, \mathcal{T}, \mathcal{L}, \mathcal{I}, \Gamma, \mathcal{E}) \rightarrow H_{meta}$$

mit H_{meta} als `HolisticEigenmodeState`.

Nicht-Zentralitaetsregel

$\mathfrak{M}$ ist kein Scheduler, kein Executor, kein Root-Account und kein zentraler Controller. $\mathfrak{M}$ darf keine lokalen Zustandsuebergaenge erzeugen. Er darf nur gueltige, vorhandene Artefakte kanonisch verdichten.

3.3 MonolithProjection

Eine lokale Monolith-Projektion ist ein Abschluss lokaler Konvergenz:

$$L_i(t) = \delta(D_i(t) \cdot \Omega_i(t) - \Theta_i(t))$$

Sie ist nur systemrelevant, wenn sie durch NCTCS-/PhaseMatrix-/Eval-Gates als tracebar, evidence-linked und replaybereit ausgewiesen wurde.

3.4 Isomorphe Feldoperator-Projektion

Ein lokaler Feldoperator U ist isomorph zur globalen Operatorik, wenn ein Report nachweist:

$$\varphi \circ \Pi_U = O_{global} \circ \iota_U$$

auf der Ebene der Artefakte, nicht als ontologische Aussage. Implementatorisch bedeutet dies:

- gleicher Operatorpfad oder kompatibles Operatorpfad-Schema;
- erhaltene Gate-Reihenfolge;
- erhaltene Trace-/Replay-Abhaengigkeiten;
- kontrollierte Abweichung der Signaturmetriken;
- keine nicht-gegatete Persistenz.

4 Kanonische Datentypen

4.1 Basistypen

```
pub type Hash256 = [u8; 32];
pub type StableId = String;
pub type Fixed = CanonicalNumber;

pub struct EvidenceRef {
    pub kind: String,
    pub address: Hash256,
    pub relation: String,
}
```

4.2 MetatronRunDescriptor

```
pub struct MetatronRunDescriptor {
  pub run_id: StableId,
  pub parent_validation_run_id: Hash256,
  pub nctcs_conformance_report_id: Hash256,
  pub canonicalization_version: String,
  pub operator_versions: BTreeMap<String, String>,
  pub thresholds: MetatronThresholds,
  pub policies: MetatronPolicies,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct MetatronThresholds {
  pub min_nctcs_class: NctcsClass,
  pub min_global_coherence: Fixed,
  pub min_isomorphism_score: Fixed,
  pub min_spectral_gap_delta: Fixed,
  pub min_replay_identity: Fixed,
  pub max_unexplained_drift: Fixed,
  pub min_eval_confidence: Fixed,
}

pub struct MetatronPolicies {
  pub require_domain_validation: bool,
  pub require_c4_for_global_eigenmode: bool,
  pub allow_diagnostic_only: bool,
  pub reject_on_missing_trace: bool,
  pub reject_on_non_materialized_inputs: bool,
}
```

4.3 HolisticEigenmodeState

```
pub struct HolisticEigenmodeState {
  pub state_id: Hash256,
  pub parent_run_id: Hash256,
  pub null_center_ref: Hash256,
  pub field_tensor_ref: Hash256,
  pub macro_control_state_ref: Hash256,
  pub trace_head: Hash256,
  pub local_monolith_refs: Vec<Hash256>,
  pub isomorphic_projection_refs: Vec<Hash256>,
  pub spectral_gap_report_ref: Hash256,
  pub aggregate_gate_status: AggregateGateStatus,
  pub global_coherence: Fixed,
  pub conformance_class: MetatronConformanceClass,
  pub validation_status: MetatronValidationStatus,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

4.4 LocalMonolithProjection

```
pub struct LocalMonolithProjection {
  pub projection_id: Hash256,
  pub source_substrate_id: Hash256,
  pub local_operator_path_hash: Hash256,
  pub resonance_product: Fixed,
  pub convergence_envelope: Fixed,
```

```

pub trigger_threshold: Fixed,
pub emitted: bool,
pub materialized_ref: Option<Hash256>,
pub trace_ref: Hash256,
pub evidence_refs: Vec<EvidenceRef>,
}

```

4.5 IsomorphicProjectionReport

```

pub struct IsomorphicProjectionReport {
pub report_id: Hash256,
pub local_operator_ref: Hash256,
pub global_operator_ref: Hash256,
pub local_path_hash: Hash256,
pub global_path_hash: Hash256,
pub operator_path_compatible: bool,
pub gate_order_preserved: bool,
pub trace_dependency_preserved: bool,
pub replay_dependency_preserved: bool,
pub signature_distance: Fixed,
pub isomorphism_score: Fixed,
pub passed: bool,
pub evidence_refs: Vec<EvidenceRef>,
}

```

4.6 SpectralGapStitchReport

```

pub struct SpectralGapStitchReport {
pub report_id: Hash256,
pub prior_gap: Fixed,
pub post_gap: Fixed,
pub delta_gap: Fixed,
pub stitch_refs: Vec<Hash256>,
pub tensor_delta_report_ref: Hash256,
pub improved_or_preserved: bool,
pub evidence_refs: Vec<EvidenceRef>,
}

```

5 Operatorpfad

5.1 Canonical Closure Chain

Der Layer verarbeitet keine Rohdaten. Er arbeitet nur auf bereits validierten Artefakten:

$ValidationRun \rightarrow NCTCSReport \rightarrow MacroControlState \rightarrow LocalMonolithProjection^* \rightarrow IsomorphicProje$

5.2 Referenzalgorithmus

```

pub fn run_metatron_closure(
rd: MetatronRunDescriptor,
input: MetatronClosureInput,
) -> MetatronClosureOutcome {
let nctcs = load_nctcs_report(input.nctcs_report_ref);
let validation = load_validation_report(input.validation_report_ref);

let preflight = check_required_inputs(&rd, &nctcs, &validation);

```

```

if !preflight.passed {
  return MetatronClosureOutcome::Rejected(preflight.into_report());
}

let local = collect_local_monolith_projections(&input);
let iso = evaluate_isomorphic_projections(&rd, &local, &nctcs);
let gap = evaluate_spectral_gap_stitching(&rd, &input);
let gates = evaluate_metatron_gate(&rd, &nctcs, &validation, &iso, &gap);

if !gates.passed {
  return MetatronClosureOutcome::Diagnostic(MetatronDiagnosticReport {
    gate_report: gates,
    local_projections: local,
    isomorphism_reports: iso,
    spectral_gap_report: gap,
  });
}

let state = build_holistic_eigenmode_state(
  &rd, &nctcs, &validation, &local, &iso, &gap, &gates
);

MetatronClosureOutcome::Closed(state)
}

```

6 Gates

6.1 Metatron Gate

$$G_{meta} = G_{nctcs} \wedge G_{trace} \wedge G_{replay} \wedge G_{iso} \wedge G_{gap} \wedge G_{eval} \wedge G_{drift}$$

Gate	Bedeutung
G_{nctcs}	NCTCS-Conformance erreicht mindestens die geforderte Klasse.
G_{trace}	Alle Eingangsdaten besitzen TraceRefs.
G_{replay}	ReplayIdentity erfuehlt Mindestschwelle.
G_{iso}	Lokale Monolith-Projektionen sind operatorisch kompatibel mit globalem Pfad.
G_{gap}	Spektrale Luecke ist erhalten oder verbessert.
G_{eval}	Eval-/Validation-Status erlaubt Closure.
G_{drift}	Kein unerklärter Drift ueber Schwellwert.

6.2 Fail-Closed-Regel

Wenn $G_{meta} = 0$, darf kein `HolisticEigenmodeState` mit produktivem Status entstehen. Erlaubt sind nur diagnostische Reports.

6.3 Coherence-Is-Not-Truth-Regel

Hohe globale Kohärenz darf nicht als Wahrheit ausgegeben werden. Sie ist nur ein Beitrag zum Gate. Wahrheit oder Nutzwert entsteht erst durch Evidence, Replay, DomainValidation und Baseline-Vergleich.

7 Conformance Classes

Klasse	Bedingung
M0	Typen und Reports sind kanonisch serialisierbar.
M1	Lokale Monolith-Projektionen werden erkannt und content-addressed gespeichert.
M2	IsomorphicProjectionReports werden erzeugt und Gate-Reihenfolge wird geprueft.
M3	SpectralGapStitchReport, Trace, Replay und Eval-Artefakte sind integriert.
M4	Ein gueltiger HolisticEigenmodeState entsteht aus NCTCS-C4 oder explizit freigegebenem Strukturstatus.
M5	M4 plus reale DomainValidation mit empirischem oder diagnostischem Urteil.

8 Integration mit Validation-Runner und Eval-Matrix

8.1 Runner-Erweiterung

Der Validation-Runner **MUST** nach Abschluss eines vollstaendigen Laufs optional oder standardmaessig den MetatronClosure ausfuehren, sofern die notwendigen Artefakte vorhanden sind.

```
pse-validation-runner run-all --domain domain_manifest.json \
  --metatron-closure \
  --out validation_runs/<commit>/
```

8.2 Neue Reports

- metatron_closure_report.json
- holistic_eigenmode_state.json
- isomorphic_projection_report.json
- spectral_gap_stitch_report.json
- metatron_gate_report.json

8.3 Neue Metriken

Metrik	Bedeutung
GlobalCoherenceScore	Aggregierte Kohärenz aus MacroControlState, Monolith-Projektionen und FieldTensor.
IsomorphicProjectionScore	Kompatibilitaet lokaler Operatorik mit globaler Operatorik.
SpectralGapDelta	Veraenderung der spektralen Luecke nach Stitches.
MonolithProjectionRate	Anteil lokaler Projektionen, die als gueltig anerkannt werden.
HolisticClosurePassRate	Anteil der Runs mit bestandenem MetatronGate.
UnexplainedDriftScore	Nicht durch Trace/Evidence erklarte globale Abweichung.
CoherenceTruthSeparation	Nachweis, dass Kohärenz nicht als alleiniger Wahrheitsindikator verwendet wurde.

9 CLI

```
pse-metatron inspect <validation_run_dir>
pse-metatron project-local <validation_run_dir> --out local_monoliths.json
pse-metatron isomorphism <validation_run_dir> --out iso_report.json
pse-metatron spectral-gap <validation_run_dir> --out gap_report.json
pse-metatron close <validation_run_dir> --out metatron_closure_report.json
pse-metatron replay <metatron_closure_report.json>
pse-metatron verify <holistic_eigenmode_state.json>
```

10 Tests

10.1 Unit Tests

1. metatron_operator_does_not_mutate_inputs
2. local_monolith_projection_is_content_addressed
3. isomorphic_projection_requires_gate_order_preservation
4. isomorphic_projection_rejects_missing_trace_dependency
5. spectral_gap_delta_is_deterministic
6. coherence_alone_does_not_pass_metatron_gate
7. metatron_gate_fails_closed_without_replay
8. holistic_eigenmode_requires_macro_control_state

10.2 Integration Tests

1. validation_run_can_emit_metatron_closure_report
2. metatron_replay_byte_identical
3. diagnostic_outcome_when_domain_validation_missing
4. m4_closure_requires_nctcs_c4_by_default
5. isomorphism_reports_sorted_before_hashing
6. metatron_verify_detects_modified_input_artifact

10.3 Negative Tests

1. Kein HolisticEigenmodeState ohne NCTCS-Report.
2. Kein produktiver Closure-Status ohne Trace und Replay.
3. Keine lokale Monolith-Projektion darf direkte Persistenz erzeugen.
4. Kein Gate darf globale Kohärenz allein als Wahrheit akzeptieren.
5. Kein nichtdeterministisches Hashing oder platform-float im Auditpfad.

11 Definition of Done

Eine Implementierung gilt als abgeschlossen, wenn:

1. alle neuen Typen implementiert und dokumentiert sind;
2. CLI-Kommandos `inspect`, `project-local`, `isomorphism`, `spectral-gap`, `close`, `replay`, `verify` existieren;
3. Reports JCS- oder projektkanonisch serialisiert werden;
4. HolisticEigenmodeState content-addressed ist;
5. MetatronOperator keine Eingangsartefakte mutiert;

6. Validation-Runner die Closure optional ausfuehren kann;
7. Eval-Matrix die neuen Metriken aufnehmen kann;
8. alle Unit-, Integration- und Negative-Tests bestehen;
9. cargo fmt, cargo clippy, cargo test -workspace und Docs sauber laufen.

12 Coding-Agent-Auftrag

PATCH-ID: PSE-METATRON-MONOLITH-01

TITLE: Holistic Eigenmode Closure Layer

Implementiere einen additiven Closure-Layer ueber dem bestehenden PSE-Stack. Der Layer darf keine neue Engine bauen und keine bestehenden Artefakte ersetzen. Er liest Validation-/Eval-/NCTCS-/PhaseMatrix-/Stitch-/TPT-Artefakte, prueft lokale Monolith-Projektionen, Isomorphie, spektrale Luecke, Trace, Replay und Eval-Status, und erzeugt bei Gate-Pass einen content-addressed HolisticEigenmodeState.

MUST:

- MetatronRunDescriptor, HolisticEigenmodeState, LocalMonolithProjection, IsomorphicProjectionReport, SpectralGapStitchReport und MetatronGateReport implementieren.
- MetatronGate fail-closed implementieren.
- Validation-Runner-Integration bereitstellen.
- Eval-Matrix-Metriken ergaenzen.
- CLI pse-metatron bereitstellen.
- Replay und Verify fuer Closure-Reports implementieren.

MUST NOT:

- keine Eingangsartefakte mutieren;
- keine neue PhaseMatrix/NCTCS-Parallelengine bauen;
- keine lokale Resonanz direkt persistent machen;
- keine globale Kohärenz als Wahrheit behandeln;
- keine nichtdeterministischen Auditpfade einfuehren.

13 Schlussinvariante

$$Metatron \neq Controller$$

$$Metatron = Eigenmode(K_0, F_T, M_{nctcs}, Trace, Eval, \mathcal{L}, \mathcal{I}, \Gamma)$$

Endformel

Ein lokaler Monolith darf als Projektion der globalen Operatorik gelten, wenn seine Operatorbahn, Gate-Reihenfolge, Trace-Abhaengigkeiten, Replay-Abhaengigkeiten und Signaturmetriken mit dem globalen Closure-Pfad kompatibel sind. Der globale Metatron-Zustand entsteht nicht als Befehl, sondern als replayfaehige Verdichtung aller gueltigen lokalen Projektionen in der persistenten Tensorhistorie.
